Ford VCM II — Internals, Recovery & Linux Modding

Nov 7, 2026 · @Jakka351

The VCM II is an embedded-Linux device, not a dumb USB-to-OBD adapter — which is why it has a recovery mode, a test-mode web server and a telnet root shell. These are field notes on the platform, the official repair/recovery path, the community test-mode shell, and the serial/module/firmware modding scene.

The platform

The VCM II (Ford/Rotunda badge, built by Bosch/ETAS on Vetronix heritage; genuine units are the ETAS **6516** family) is Ford's OEM vehicle communication module — the pass-thru interface IDS drives to reach Ford/Lincoln/Mercury and Mazda vehicles over the 16-pin DLC.

Inside, it's a small **embedded Linux** box (BusyBox-style userland, `bash-3.2`), and that is the whole reason this document exists: it boots a kernel, mounts a root filesystem off NOR flash + the SD card, and runs a web server and a telnet daemon. Recovery mode, test mode and the root shell all hang off that.

Physical landmarks

- **Recovery Mode Switch** — a plastic tab under the rubber boot on the end *opposite* the DLC connector.
- **microSD slot** — under the boot on the USB side. The card carries filesystem/config data and is **paired to the unit's serial number**; drop it in another VCM II and it won't run.
- **Front DC power port** — feed it from a bench supply during firmware work; external power is repeatedly what makes serial/type/firmware writes actually take.
- **Internal HDL26-PL-B connector** — the **serial console / bootloader** lives on **pins 24 & 25** (`ttyS0`, 38400 8N1).
- **Connectivity** — wired USB plus an optional **WiFi** add-on card. Wired USB is mandatory for loading firmware and for CFR setup.

Hardware revs in the wild: genuine CCA is commonly **REV D**; clones are often **REV C**, sometimes with a substituted **U54** chip, hot-glued jacks, or an unsoldered red "banana" connector. Clone PCBs vary enough that two "identical" units can behave differently on

the same IDS version.

Flash layout & boot chain

The on-board NOR flash is carved into MTD partitions; the bootloader passes this `mtdparts` line to the kernel (lifted from a recovery image, so sizes/names are authoritative):

```
root=/dev/ram init=/linuxrc console=ttyS0,38400 rw ramdisk_size=24576
mtdparts=flash0:256k(fsboot),256k(boot),128k(bootvars),640k(fpga),
1536k(linux2),6144k(initrd2),3072k(linux1),19328k(initrd1),
1024k(jffs2),128k@32384k(linuxvars1),128k(linuxvars2),128k(snmac)
board_id=0x1
```

Device	Name	Size	Role
mtd0	fsboot	256k	first-stage boot — **do not touch**
mtd1	boot	256k	bootloader (U-Boot) — **do not touch**
mtd2	bootvars	128k	boot environment
mtd3	fpga	640k	FPGA bitstream
mtd4	linux2	1536k	**recovery** kernel
mtd5	initrd2	6144k	**recovery** ramdisk
mtd6	linux1	3072k	production kernel
mtd7	initrd1	19328k	production ramdisk
mtd8	jffs2	1024k	RW config, mounted `/etc/vci/config`
mtd9	linuxvars1	128k @0x7E80000	Linux env vars
mtd10	linuxvars2	128k	Linux env vars
mtd11	snmac	128k	**serial number / MAC**

What this tells you

- Boot path: `fsboot` → U-Boot (`mtd1`) → kernel (production `linux1`/`initrd1`, or recovery `linux2`/`initrd2`) → Linux. The `init=/linuxrc` + `root=/dev/ram` means it boots a RAM disk, not the SD card — the SD is data/config, not rootfs.

- `ttyS0` @ 38400 8N1 is the console, the same line broken out on the HDL26-PL-B pins.
- The unit's **serial number and module type resolve to the snmac region (`mtd11`)**. That's the partition the serial-change tricks write — and newer IDS firmware deliberately **shifts the `mtd11` offset** so the old rewrite lands in the wrong place. Clever, and the main reason re-serialising fails on late firmware.
- `mtd4` + `mtd5` are the **recovery firmware pair**; a custom recovery image replaces exactly these two and leaves the production kernel alone.

Recovery / Repair Mode (the official un-brick)

This one is sanctioned and documented — it's in the Ford VCM II Hardware Manual and dealer TSBs. Reach for it when a firmware update failed, the SD is corrupt, the Error LED is red at boot, or the unit is stuck beeping once a second in a boot loop.

1. Unplug the DLC cable from the vehicle.
2. Unplug the USB cable from the VCM II and the PC.
3. Pull the rubber boot on the end **opposite the DLC connector** to expose the Recovery Mode Switch (the plastic tab).
4. Launch IDS (and/or the VCI Manager / "VCM II Manager").
5. **Press and hold** the tab — don't release until step 8.
6. With the tab still held, connect the USB cable to the PC.
7. Keep holding until the **Power LED stays solid** and the unit **beeps** (~20–30 s).
8. Release the tab → the unit is in **Recovery Mode**; IDS / VCI Manager pops the "new firmware available / Recover" dialog.
9. Click **Recover** → **Start Update**. Do **not** pull power or USB during the write.
10. It rewrites the firmware, reboots, and beeps when done. Relaunch IDS; the normal icon means it recovered.

Field notes

- **Power the unit from the front DC port** off a bench supply for any firmware op. Repeated reports that type/serial/firmware changes only succeed on external power — the USB rail sags mid-write otherwise and the change silently fails or reverts.
- Manager builds people lean on: **2.1.1.5** and **2.1.119.3** (a.k.a. 2.119.x). IDS **v86** is the "known-good baseline" a lot of procedures assume, because it predates the clone-blacklist firmware.

- A recover drops you to a stock recovery firmware (≈ FW **2.1.1.5** after an IDS 86 recover); IDS then offers to push current production firmware on top.
- The genuine SD is paired to the serial — **image the original SD before you do anything** and keep the original safe.

Test mode & the root shell

Recovery mode is the sanctioned door. **Test mode** is the community one: it brings up a diagnostic web server and a **passwordless telnet root login** on a link-local interface. You need your own unit and a microSD card reader.

Arming test mode (via the self-test hook on the SD card):

1. Recover the unit on **IDS v86** first so it sits at a known FW (≈ 2.1.1.5). Wait ~90 s after the beep so the SD repopulates.
2. Power off, eject the SD, mount it on a Linux box.
3. Edit `apps/vci-diags/hwtest-scripts/self-test.sh` and add a line:

   ```
   touch /etc/vci/config/testmode
   ```

4. Unmount cleanly, reinsert the SD, plug the VCM II into the PC, wait ~90 s past the beep.
5. Trigger the self-test once via IDS: **System Utilities → Run System Diagnostics → Test a VCM II Module → Next**, then unplug, wait a few seconds, replug.

Reaching the interfaces — set your PC to a static IP on the `192.168.171.x` subnet (e.g. `.1`, mask `/24`):

- **Web server:** `http://192.168.171.2`, ~15 s after the beep. Menu includes *Set New Serial Number*, *Set Repair Test Mode* (stay in test mode next boot), and — on modded firmware — *Set Module Type* and a web firmware update.
- **Root shell:** `telnet 192.168.171.2`, ~10–15 s after the beep (PuTTY/ExtraPuTTY, connection type Telnet, port 23). You land straight on a root prompt (`bash-3.2#`), **no credentials** on stock test-mode firmware.
- *Set Repair Test Mode* must be re-armed each boot unless you persist it (the self-test hook above, or an always-testmode custom firmware).

Image the flash first — from the root shell, dump to the SD "VCM Backups" partition:

```
dd bs=512 if=/dev/mtd0 of=/mnt/sd/mtd0
dd bs=512 if=/dev/mtd1 of=/mnt/sd/mtd1
dd bs=512 if=/dev/mtd2 of=/mnt/sd/mtd2
# ...repeat for every mtdN you want (mtd0 through mtd11)
```

Power down, copy the `mtd*.bin` files somewhere safe, and you have a known-good image. Restore is the reverse — `flash_unlock` then `flashcp` — e.g.:

```
/usr/local/mtd/flash_unlock /dev/mtd11
flashcp -v mtd11-128k.new /dev/mtd11
```

Golden rule: when restoring, **skip `mtd0`, `mtd1`, `mtd2`** (the boot partitions) unless you have serial-console recovery ready. A bad write there with no console is a hard brick — everything else is recoverable from the shell.

The modding landscape

What the scene has built on top of test mode + a root shell:

- **Serial-number change** — from the web server's *Set New Serial Number*, or by writing the `snmac` / `mtd11` region directly. The well-known public clone serials (e.g. `1211-31605352`, `31606235`) get blacklisted by newer IDS, so units are re-serialised to an un-banned number. The default stock serial after a recover is `1211-31605352`.
- **Module-type change** — flip the unit's declared identity (Ford ↔ Mazda ↔ Suzuki `6516`, etc.) so one interface drives multiple OEM apps (SDT II, Mazda IDS, JLR). Stock test-mode can do it; the common modded firmware exposes it under *Other Functions → Set Module Type*. Known gotcha: **Mazda IDS reverts the serial/type to original** on its own VCI update — the change sticks under Ford, then gets stomped the moment you plug into Mazda IDS.
- **Custom recovery firmware (the "ColtB45" image)** — the canonical modded recovery build. It keeps every stock test-mode function and adds a telnet server, an FTP server, the test-mode web server always-on, module-type change in the web UI, and WiFi in recovery (interface left disabled by default). It mounts the SD RW at `/mnt/sd` and the JFFS2 config RW at `/etc/vci/config`, and ships with a **known fixed root password** in place of the passwordless stock shell. It's installed by pushing the recovery kernel/ramdisk images (`mtd4.new` / `mtd5.new`) into `/tmp` over FTP/telnet and

`flashcp`-ing them onto `mtd4`/`mtd5` — so it only rewrites the recovery pair and leaves production boot intact. Later custom installers add a "drop to a Linux shell" option outright.

Why anyone does this — three honest buckets:

- **Repair / RE (clean):** un-brick a dead unit, image your own hardware, learn the platform, recover over the console. Defensible, all on gear you own.
- **Cross-OEM use:** running one interface as different module types to drive multiple manufacturers' software.
- **Blacklist evasion / licensing:** most serial-spoofing exists so counterfeit interfaces keep running licensed OEM software after Ford bans their serials. That's licence circumvention whatever the hardware is — legally a different thing from the first two buckets, and worth naming as such rather than pretending it's all "repair."

Clones & the IDS blacklist cat-and-mouse

- **Genuine vs clone.** Genuine = Bosch/ETAS `6516` CCA, commonly **REV D**. Clones vary wildly: often **REV C**, substituted **U54** chip, hot-glued DC jack, an unsoldered red "banana" connector. Some clones keep working as late as IDS 122; others stop after IDS 96. Two clones off the same listing can behave differently on the same IDS build.
- **The ban.** Ford/Bosch ship firmware through IDS that blacklists known clone serials — clones are *most likely banned from v92.05 onwards*. Once banned, IDS either refuses the unit or tries to force a firmware update that bricks or blocks it.
- **Community counter-moves:**
  - **Freeze the VCI firmware folder** — replace the firmware folder under `C:\Program Files\Bosch\...` (or the x86 path) with an older **2.115** or **2.119** set so IDS can't push the banning firmware. If files are locked, kill the `vtxi` / launcher process in Task Manager first. This is the single most common clone fix.
  - **Run old IDS** (v86–v91) where clones still enumerate.
  - **Re-serialise** to a non-blacklisted number (see modding).
  - **Pin firmware per app:** JLR tends to want 2.115; Mazda 94xx runs on 2.119 — dual setups keep both.
- **The counter-counter-move.** Newer firmware shifts the `mtd11` offset so the old serial rewrite misses, and Mazda IDS rewrites the serial back to stock on update. External DC power on the front port is the repeatedly-cited trick for making a firmware/type/serial change actually stick against this.

Net: a running arms race. The stable config most people settle on is a clone **frozen on 2.119, never allowed to "update"** online.

Bricking & recovery of last resort

- **The classic brick.** Running `cgi-bin/total-reflash` from the test-mode web server wipes the unit. Don't poke CGI endpoints blind — that one is a trap people have fallen into.
- **Boot-partition brick.** A bad write to `mtd0` / `mtd1` / `mtd2` with no console means no Linux, no telnet, nothing to recover *from* in software.

Last-resort ladder (easiest → hardest):

1. **Still have a shell, just bad bootvars?** Edit the boot environment from within Linux and reboot — no console hardware needed.
2. **Serial console** on HDL26-PL-B **pins 24 & 25** (`ttyS0` , 38400 8N1) → drop to U-Boot, fix `bootargs` , tftp a kernel, re-flash. This is the real recovery path once Linux won't boot but the bootloader is alive.
3. **Bootloader itself gone?** JTAG, or desolder and read the NOR chip, reprogram it from a known-good dump with an external programmer — or clone the chip from a working unit.

Habits that keep you out of the hole:

- Image **before** you touch anything; label which `mtdN` is which.
- Never restore boot partitions casually — `mtd0` / `mtd1` / `mtd2` are the ones that cost you a programmer.
- Keep external DC power on the front port during every write.
- Keep the original paired SD card safe and untouched.

Sources & further reading

Mostly community forums — treat specifics as field-sourced and verify on your own unit before trusting them. As of Nov 2026, the MHH Auto thread is still the canonical reference.

- MHH Auto — "VCM2 Hacking" thread — the canonical source: ColtB45 test-mode/root-shell method, MTD layout, `dd` backups, custom recovery firmware, serial & module-type changes.

- MHH Auto — IDS v93/v94 clone VCM2 firmware fix — the 2.115 / 2.119 firmware-freeze fix.
- fordvcm2 (obdexpress mirror) — change serial, enter test mode, get a root shell — the step-by-step test-mode + telnet-root write-up.
- Ford VCM II Hardware Manual (ManualsLib) — official recovery mode, LED/error states, specs, operating range.
- forddoctorsdts — VCM code update (TSB steps) — the official recovery-mode procedure.
- obdii365 — VCM II firmware upgrade & downgrade — VCI Manager recovery flow.
- Suzuki-forums — VCM2 → 6516-Suzuki (SDT II) — module-type swap in practice.

Scope note: this is reference/repair-and-RE material on hardware you own. Re-serialising to dodge a blacklist so a clone can run licensed OEM software is licence circumvention — called out in "The modding landscape" — and is a separate question from the recovery and platform work.